



Experimentos com CIFAR10

Antonio Alberto Moreira de Azevedo

Programa de Engenharia Elétrica
Instituto Alberto Luiz Coimbra de Pós-Graduação e Pesquisa de Engenharia
Universidade Federal do Rio de Janeiro

16/09/2025

Sumário

1. Introdução

2. Conjuntos de Dados

3. Modelos

4. Experimentos

5. Resultados dos Experimentos e conclusão

Introdução

Introdução

Objetivo do trabalho:

- Desenvolvimento de pipeline de classificação de imagens empregando os modelos **ResNet18 (*baseline*)** [1], **Capsule Networks (CAPSNET)** [2] e o **Vision Transformer (ViT)** [3] [4] com o objetivo de compara-los.
- Escolha da base de dados **CIFAR10**.

Conjuntos de Dados

CIFAR10

Este *dataset* foi utilizado para desenvolvimento dos experimentos para tarefa de classificação de imagens.

- **60.000** imagens coloridas.
- 10 classes, **6.000** imagens por classe.
- 50.000 para treino, 10.000 para teste.
- dimensões de: 32x32x3.
- Avião, Automóvel, Passáreo, Gato, Cervo, Cachorro, Sapo, Cavalo, Navio e Caminhão.

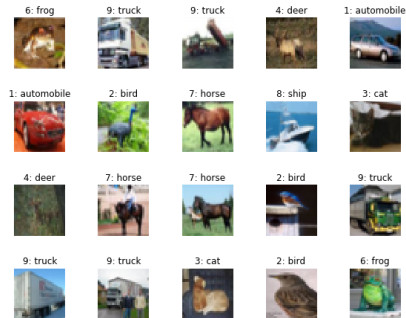


Figura: Amostra do CIFAR10

Modelos

ResNet18 (baseline)

O que é a ResNet18?

- ResNet (Residual Network): Arquitetura proposta em 2015 (He et al., Deep Residual Learning for Image Recognition);
- ResNet18: versão com 18 camadas, considerada leve e eficiente;
- Introduz o conceito de "skip connections"(atalhos residuais) → permitem que o gradiente flua melhor durante o treinamento, evitando vanishing gradient;
- Muito utilizada em classificação de imagens (ex: CIFAR10, ImageNet).

ResNet18 (baseline)

Principais características

- Estrutura: blocos residuais compostos por;
 - Convoluções 3×3 ;
 - Normalização em batch;
 - Função de ativação ReLU;
 - Conexão residual (entrada somada à saída).
- Benefícios:
 - Treinamento de redes mais profundas e estáveis.
 - Boa acurácia com baixo custo computacional.
 - Versatilidade: serve como backbone em muitas aplicações (detecção, segmentação).
- ResNet18 x ResNet34/50: ResNet18 é mais rápida e leve, indicada para bases menores ou hardware limitado.

Arquitetura

- Camada convolucional inicial.
- **Primary Capsules** (lower-level): vetores derivados de filtros convolucionais.
- **Digit Capsules** (higher-level): uma por classe (ex: dígitos 0-9).
- Classificação baseada na magnitude do vetor de saída.

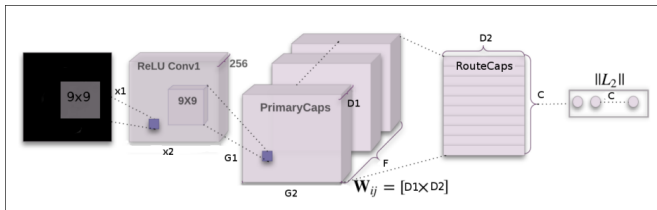


Figura: Arquitetura da CapsNet

Capsnet

Resumo da Implementação

- Baseada no artigo *Dynamic Routing Between Capsules* [2].
- Framework: PyTorch.
- CIFAR-10 (com adaptações).
- Treinamento com **perda de margem + decoder de reconstrução**.

Arquitetura:

1. Conv1 Layer: convolução inicial para extrair características.
2. Primary Capsules: vetores de baixa dimensão (8D/32D).
3. Digit Capsules: uma cápsula 16D por classe (10 para CIFAR-10), roteamento dinâmico.
4. Decoder: MLP que reconstrói a imagem original (32x32x3).

Linha do tempo de artigos – do Attention ao ViT:

- 2014 – Bahdanau attention [5] → Introduz atenção em tradução (NMT).
- 2015 – Luong attention [6] → Torna atenção prática e eficiente (global vs local).
- 2015 – Memory Networks → Usa atenção para acessar memória externa.
- 2016 – Hierarchical Attention → Atenção em diferentes níveis (texto longo).
- 2017 – Attention is All You Need [7] → Generaliza e transforma atenção no núcleo da arquitetura Transformer.
- 2017 – Attention is All You Need → Transformer no NLP.
- 2018 – BERT e GPT → popularização e pré-treinamento em larga escala.
- 2018/2019 – Non-local NN e Stand-Alone Self-Attention → primeiras aplicações em visão.
- 2020 – Vision Transformer (ViT) [3] → Transformer puro em imagens.

ViT timm

Implementação PyTorch Image Models (timm), Ross Wightman (2019-) [4]

- Biblioteca open-source em **PyTorch** para modelos de Visão Computacional.
- Função central:
 - `timm.create_model(model_name, pretrained, num_classes, ...)`
 - Criação rápida de modelos com ou sem **pesos pré-treinados**.
- Facilita experimentação em **classificação, feature extraction e fine-tuning**.

ViT timm

Recursos e Importância da implementação escolhida no Projeto

- **Treinamento eficiente:** integra AMP (mixed precision), dataloaders e augmentations.
- **Consistência:** padroniza criação de modelos → evita retrabalho.
- **Flexibilidade:**
 - Suporte a diferentes tamanhos de entrada.
 - Customização das camadas finais.
- **Relevância no trabalho final:**
 - Utilizado para criar e treinar o **Vision Transformer (ViT)**.
 - Permite **comparação direta** com CapsNet e ResNet18 no CIFAR-10.

Experimentos

Metodologia

- Escolha da Acurácia como medida de desempenho dos classificadores
- Busca de hiperparâmetros, com *k-fold*, do otimizador (Adam, RMSprop e SGD) e da Taxa de aprendizagem (entre $1e-5$ e $1e-1$). e
- treino/teste do modelo final com os melhores parâmetros por *Holdout*.
- **Hiperparâmetros padrão para os 3 modelos:**

-
- 5 folds durante a busca de hiperparâmetros
 - 12 épocas por trial (busca Optuna)
 - 192 de tamanho no batch no trial
 - 20 trials
-

-
- 50 épocas no treino final
 - 128 de tamanho do batch no treino/validação final
 - 0.1 de *holdout* final
-

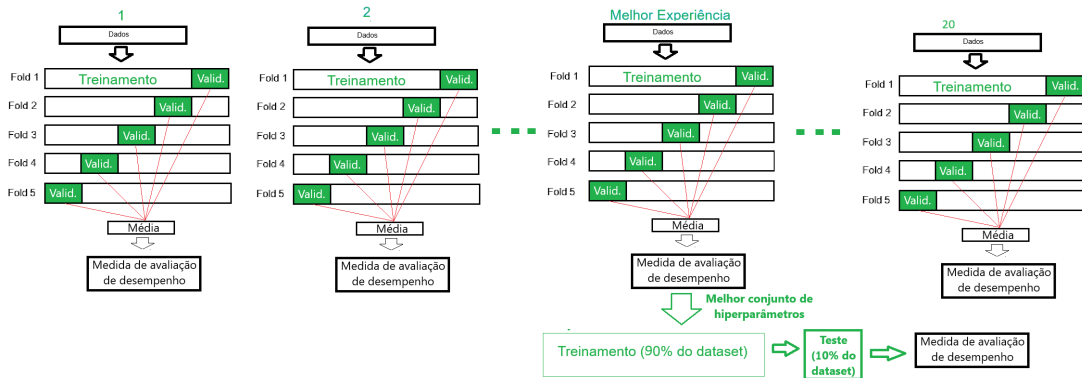


Figura: Arcabouço (pipeline) dos modelos utilizados no treinamento

Problemas encontrados

Em CPU:

- Restrições de rodar os modelos de *deep learning* com *datasets* de imagens em CPU;
- uso de memória de GPU (bytes por tensor) e *throughput* na GPU;

De treinamento: underflow (o gradiente pode ficar muito pequeno e virar zero)



Restrições do Colab: desconexão por perda de link **e tempo máximo de conexão (24h)**

Solução

- interrupção da busca (`-- timeout_optuna`) ao atingir 79.000s (21,4h).
- Uso de Automatic Mixed Precision (AMP):
 - Diz ao PyTorch para executar certas operações (como convoluções) em FP16 (16-bits) quando for seguro.
 - Algumas operações críticas (softmax, batchnorm, reduções de loss) continuam em FP32 para não perder estabilidade numérica.
 - reduz uso de memória de GPU (menos bytes por tensor) e aumenta throughput nas GPUs (que têm núcleos Tensor otimizados para FP16).
- Uso do Gradescaler:
 - Em FP16, pode haver “underflow”.
 - O scaler multiplica dinamicamente o gradiente por um fator antes do backward, e depois divide de volta na atualização de pesos.

Resultados dos Experimentos e conclusão

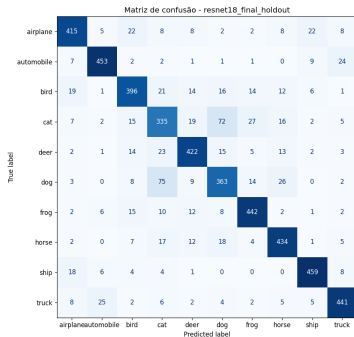
Hiperparâmetros e Métricas

Tabela: Hiperparâmetros e Acurácia (Acc) por Modelo

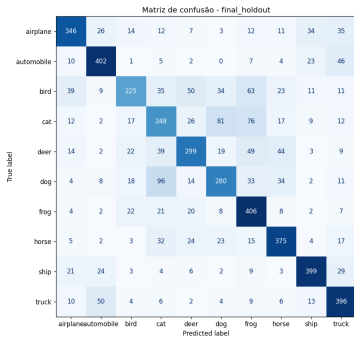
Modelo	Acc média da busca	Otimizador	Taxa	Acc treino final
ResNet18	0,768	Adam	0,0026051	0,837
CapsNet	0,766	RMSprop	0,0002363	0.678
ViT	0,682	Adam	0,0002027	0,743

Matrizes de confusão dos Treinos finais

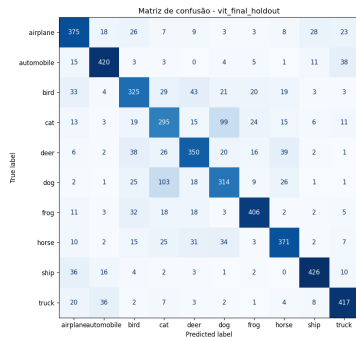
ResNet18 (baseline)



CapsNet



Vit timm



Conclusão

Este estudo analisou os modelos de classificação de imagens REsNet18, CapsNet e ViT timm empregando o dataset CIFAR10.

Os modelos CAPsnet e Vit timm não conseguiram superar o ResNet18 no resultado de classificação.

Recomendações para futuras pesquisas:

- Acelerar a busca com reporte de acurácia de validação por época para decisão da poda - habilitando *pruning* de trials ruins via *MedianPruner*.

References I

- [1] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- [2] S. Sabour, N. Frosst, and G. E. Hinton, “Dynamic routing between capsules,” *Computer Science > Computer Vision and Pattern Recognition*, 2017.
- [3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

References II

- [4] R. Wightman, “Pytorch image models.”
<https://github.com/huggingface/pytorch-image-models>, 2019.
- [5] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.
- [6] M.-T. Luong, H. Pham, and C. D. Manning, “Effective approaches to attention-based neural machine translation,” in *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1412–1421, 2015.

References III

- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS)*, pp. 6000–6010, 2017.



Obrigado

Antonio Alberto Moreira de Azevedo

Programa de Engenharia Elétrica

Instituto Alberto Luiz Coimbra de Pós-Graduação e Pesquisa de Engenharia

Universidade Federal do Rio de Janeiro

16/09/2025